

Теоретический конспект №1

Тема: Введение в Deep Reinforcement Learning

1. Что такое Reinforcement Learning (RL)

Reinforcement Learning (обучение с подкреплением) — раздел машинного обучения, в котором **агент** взаимодействует со **средой (environment)**, выполняя **действия (actions)**, чтобы **максимизировать вознаграждение (reward)**, получаемое со временем. Главная идея:

«Учиться на собственном опыте через пробу и ошибку.»

Цикл Agent–Environment

1. Агент находится в состоянии среды s_t
2. Он выбирает действие a_t
3. Среда возвращает:
 - новое состояние s_{t+1}
 - и вознаграждение r_{t+1}
4. Агент обновляет стратегию $\pi(a|s)$, чтобы **максимизировать суммарное вознаграждение**.

$$s_t \xrightarrow{a_t} (r_{t+1}, s_{t+1})$$



Цикл Agent–Environment

Марковский процесс принятия решений (MDP)

Интуитивно: среда описывается состояниями, действиями, вероятностями переходов, вознаграждениями и дисконтированием. Полная формальная запись и разбор компонентов (включая связь $r(s, a) = \mathbb{E}[R_{t+1} \mid s, a]$) — в [note 2.md](#).

Цель агента

Максимизация **ожидаемой суммы дисконтированных наград**:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

Найти **оптимальную политику** $\pi^*(a|s)$, такую что:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi}[G_t]$$

2. Почему Deep Reinforcement Learning?

Классическое RL использовало таблицы (например, Q-таблицы), но при больших пространствах состояний это невозможно. Решение — **глубокие нейронные сети**, которые аппроксимируют функции ценности или политики.

Классический RL	Deep RL
Табличные методы	Нейронные сети
Простые среды	Сложные задачи (видео, роботы)
Ограниченная масштабируемость	Высокая обобщающая способность

Примеры трансформации:

- Q-Learning → **Deep Q-Learning (DQN)**
- Policy Gradient → **Deep Policy Gradient**
- Actor-Critic → **Deep Actor-Critic**

Подробнее о том, как и почему работают нейронные сети в RL: см. [note 5.md](#)

3. Пример: LunarLander

Среда LunarLander-v2 из **Gymnasium**.

Задача: посадить лунный модуль между флагами, используя минимальное топливо.

Состояние s_t :

- координаты (x, y)
- скорости (vx, vy)
- угол и угловая скорость
- флаги касания (0/1)

Действия a_t :

1. ничего не делать
2. включить левый двигатель
3. включить главный двигатель
4. включить правый двигатель

Вознаграждения:

- +100...+140 — успешная посадка
- -100 — крушение
- -0.3 — за использование топлива
- +10 — за касание без крушения

4. Stable-Baselines3: реализация PPO

```
import gymnasium as gym
from stable_baselines3 import PPO

env = gym.make("LunarLander-v2")
model = PPO("MlpPolicy", env, verbose=1)
model.learn(total_timesteps=200_000)
model.save("lunar_lander_agent")
```

5. Что ты освоишь

- Поймёшь фундаментальные принципы RL
 - Запустишь первую среду (LunarLander-v2)
 - Обучишь агента с помощью PPO или DQN
 - Проверишь его поведение
 - Загрузишь модель на Hugging Face Hub
-

6. Гипотеза вознаграждения (Reward Hypothesis)

Reward Hypothesis — центральная идея RL:

Любую цель можно описать как задачу **максимизации ожидаемого суммарного вознаграждения**.

Почему это важно?

Обучение с подкреплением основано на фундаментальной идее: любую целевую задачу — будь то обучение ходить, играть в игры, управлять роботом или торговать на бирже — можно представить как максимизацию ожидаемого суммарного вознаграждения.

Ключевые следствия:

- **Единая математическая основа** для всех задач RL
- **Отсутствие прямых меток** — агент сам формирует стратегию через опыт
- **Гибкость** — изменив награды, можно полностью изменить поведение агента

Даже если внешне цель агента сложна (например, "посадить лунный модуль"), в терминах RL она формулируется как "максимизировать суммарное вознаграждение от действий".

Подробное объяснение: математическая формализация, примеры и детальный анализ
→ см. раздел 4 в [note 2.md](#)

Основано на:

- Sutton & Barto, *Reinforcement Learning: An Introduction* (2020)
- Andrea Lonza, *Алгоритмы обучения с подкреплением на Python* (2020)
- Hadelin de Ponteves, *AI Crash Course* (2019)
- RL Theory Book (Forts & Mills, 2022)